

System 17: Splitwise (Expense Sharing & Debt Simplification Engine)

Preceding Bridge: In [System 08: API Rate Limiter](#) and [System 14: Pub/Sub Message Broker](#), you mastered concurrency, state isolation, and data pipelines. In this chapter, we tackle one of the most frequently asked LLD problems at Uber, Amazon, and fintech companies: **Splitwise**, featuring dynamic expense split strategies and the **Min-Cash-Flow Debt Simplification Algorithm**.

1. Plain-English Jargon Demystifier

Technical Term	Plain English Translation	Real-World Metaphor
Split Strategy	The mathematical rule used to divide a bill among group members.	Deciding whether everyone splits 50/50, pays for what they ordered, or splits by percentage.
Net Balance	Total money paid by a user minus total money owed by that user across all expenses.	A running ledger showing if you are overall owed money (+) or owe money (-).
Pairwise Debt	Direct 1-to-1 debt tracking between two specific people ($A \rightarrow B$).	An IOU note stuck to the fridge: "Alice owes Bob \$15".
Debt Simplification	Minimizing the total number of bank transactions needed to settle all debts in a group.	Alice owes Bob 10, <i>Bob owes Charlie</i> 10. Instead of 2 transactions, Alice gives \$10 directly to Charlie!
Min-Cash-Flow Algorithm	A greedy algorithm using two priority queues (debtors and creditors) to clear all debts in at most $N - 1$ transactions.	An accountant gathering everyone in a circle and clearing the biggest debtor with the biggest creditor.

2. Spoon-Fed Mental Model: The Dinner Bill & The Roommate Whiteboard

Imagine 4 roommates go out to dinner:

- **Alice** pays the \$100 grocery bill.
- **Bob** pays the \$60 electricity bill.
- **Charlie** pays the \$40 internet bill.
- **David** pays \$0.

The Naive Pairwise Settlement (Junior Approach):

If you calculate pairwise debts:

- David owes Alice 25, *Bob*15, Charlie \$10.
- Charlie owes Alice 10, *Bob*5...
- Bob owes Alice \$10... There are **6 separate awkward Venmo transactions** floating between roommates! People are sending money back and forth unnecessarily.

The Debt Simplification Engine (Staff Approach):

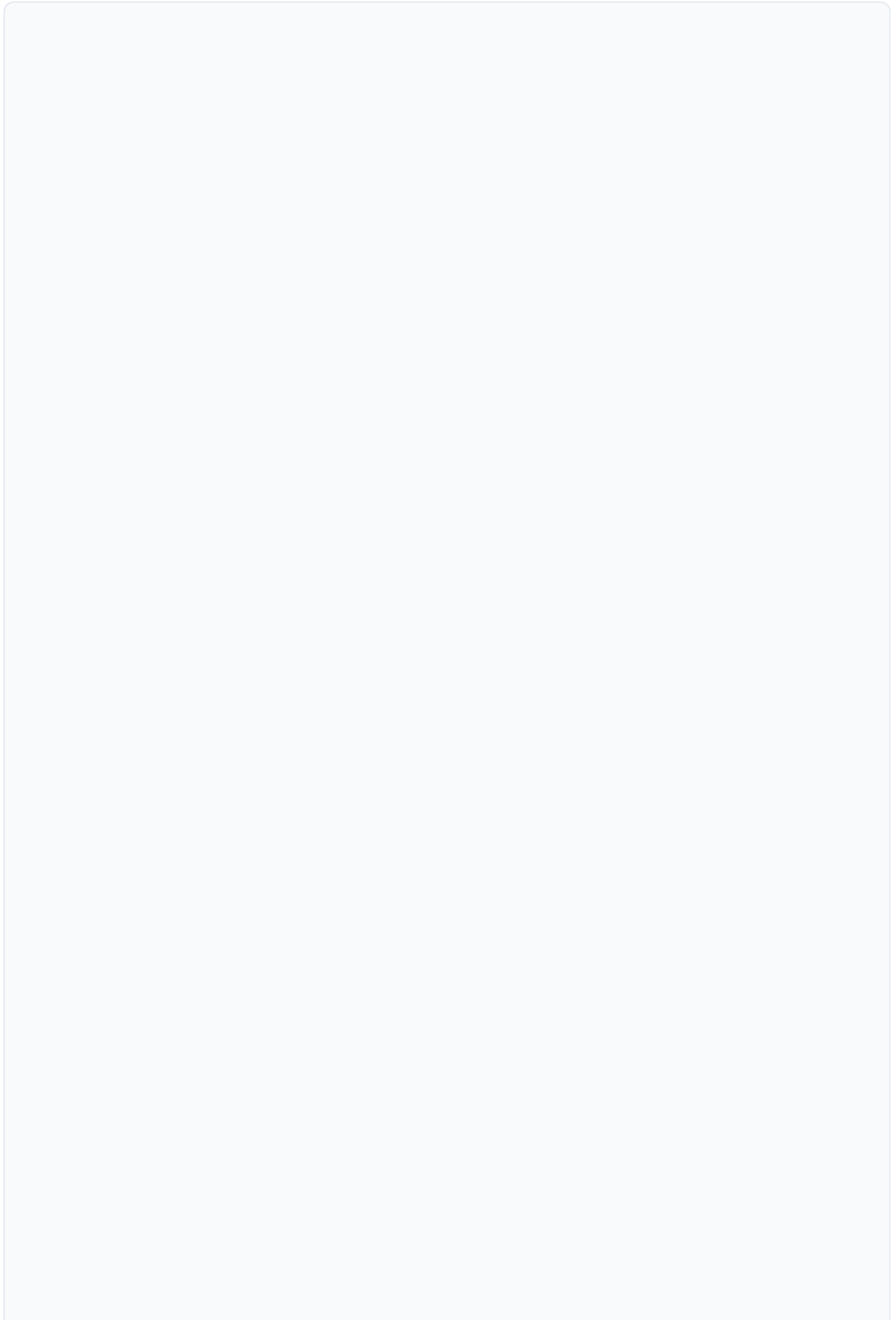
Instead of tracking who paid for what specific dish, calculate each person's **Net Balance**:

- Total spent: $100 + 60 + 40 = 200$. Equal share = \$50 each.
- **Alice**: Paid 100, *owes*50 → **Net: +\$50** (Creditor)
- **Bob**: Paid 60, *owes*50 → **Net: +\$10** (Creditor)
- **Charlie**: Paid 40, *owes*50 → **Net: -\$10** (Debtor)
- **David**: Paid 0, *owes*50 → **Net: -\$50** (Debtor)

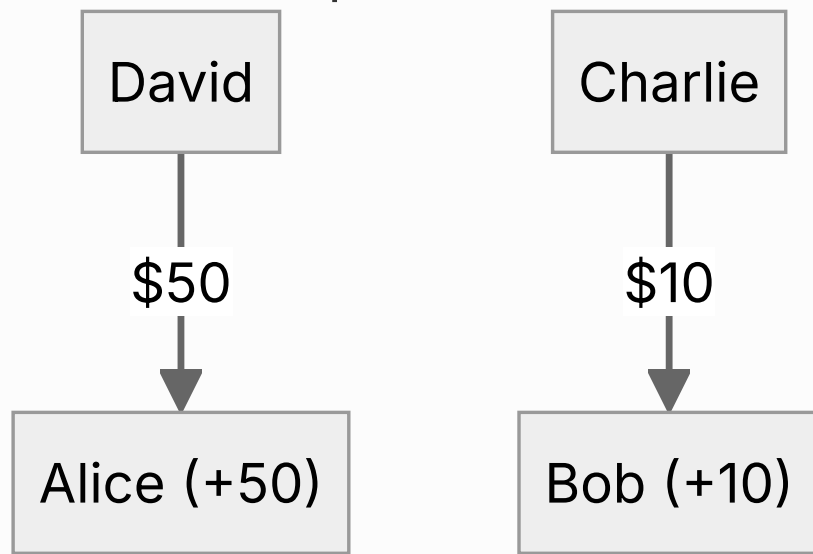
Simplified Settlement (Only 2 Transactions!):

1. David pays Alice \$50. (Alice is fully settled, David is fully settled).
2. Charlie pays Bob \$10. (Bob is fully settled, Charlie is fully settled).

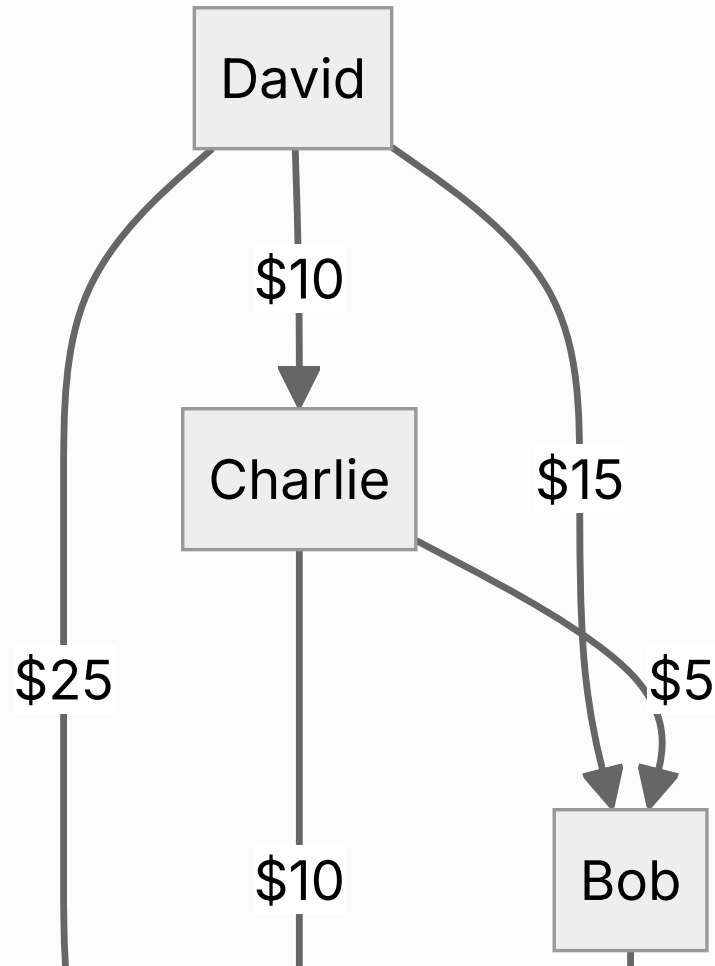
Total transactions dropped from **6 down to 2!**

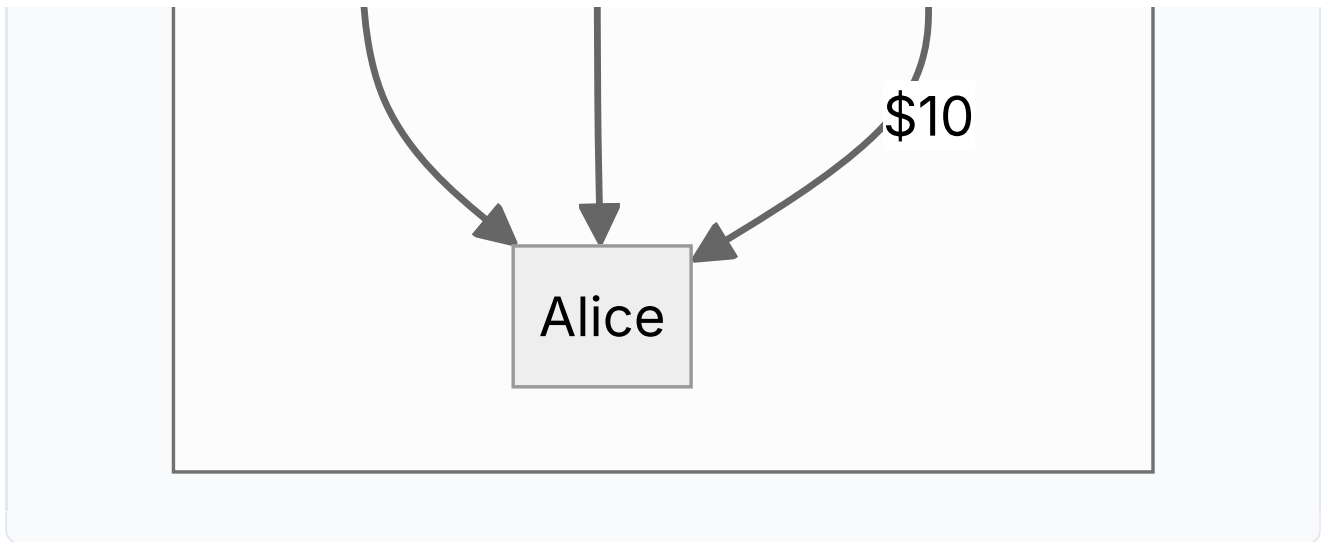


Min-Cash-Flow Simplification (2 Transactions)



Naive Pairwise Debts (6 Transactions)





3. Core Architecture & Design Patterns

Applied Patterns:

1. **Strategy Pattern** (`SplitStrategy`): Decouples the mathematical logic of splitting bills (`EQUAL` , `EXACT` , `PERCENT`) from the expense entity.
2. **Repository Pattern** (`ExpenseManager`): Centralizes in-memory storage of users, groups, and transactions.
3. **Greedy Priority Queue Pattern** (`DebtSimplifier`): Uses max-heaps to solve Min-Cash-Flow in $O(N \log N)$ time.

4. Junior vs Staff Implementation

5. Complete Runnable Implementation

Here is a 100% runnable, fully typed production implementation with unit tests:

6. Chapter Milestone Check

Verify your understanding before continuing:

1. Why is the Strategy pattern ideal for expense splitting in Splitwise?

- *Answer:* It allows new mathematical split strategies (e.g., share weights, currency conversions, custom adjustments) to be added without modifying the core `Expense` or `SplitwiseGroup` classes (adhering to Open/Closed Principle).

2. What problem does the Min-Cash-Flow algorithm solve, and what data structures are optimal for it?

- *Answer:* It eliminates redundant circular transactions between group members. It calculates each user's net balance and uses two Max-Heaps (one for creditors, one for debtors) to iteratively match the largest debtor with the largest creditor in $O(N \log N)$ time, settling all debts in at most $N - 1$ steps.

3. How do you prevent penny-rounding discrepancies when splitting 100threeways(33.33 \times 3 = \$99.99)?

- *Answer:* You calculate the base share (`round(total / N, 2)`) and assign the residual difference (`total - allocated` , here 0.01) to the first participant's share.